

Vectors (std::vector)

O "Porquê": Arrays vs. Vectors

Até agora, vimos que os arrays simples são, num geral, estruturas de tamanho fixo. Isso significa que o computador reserva um bloco único de memória logo no início e esse espaço não pode ser alterado. Se você definiu um array para 10 leituras de sensor e o robô precisar de 11, o programa travará ou lerá lixo da memória.

O `std::vector` resolve esse problema sendo um array dinâmico. Ele gerencia a memória automaticamente: quando você adiciona um elemento e o espaço atual acaba, o `vector` "se expande" (cria um novo bloco maior e move os dados). Na robótica, isso é essencial para armazenar dados que não sabemos o volume final, como o histórico de uma trajetória ou uma lista de objetos detectados por uma câmera.

Declaração e Métodos Essenciais

Para usar vectors, precisamos incluir a biblioteca `<vector>`. Diferente dos arrays, os vectors possuem funções embutidas (métodos) que facilitam muito a manipulação.

Principais Métodos:

- `push_back(valor)` : Adiciona o valor ao final da lista, aumentando o tamanho automaticamente.
- `size()` : Retorna quantos elementos existem de fato.
- `.at(indice)` : Acesso seguro. Se você tentar acessar um índice que não existe, o C++ interrompe o programa com um erro claro, em vez de ler lixo de memória como o `[]`.

```
In [ ]: %%writefile cpp_files/primeiro_exemplo.cpp
#include <iostream>
#include <vector> // Necessário para usar vectors
using namespace std;

int main() {
    vector<int> leituras_bateria;
    int entrada;

    cout << "--- Registro de Bateria ---" << endl;
    cout << "Digite os níveis (0 a 100) ou -1 para encerrar:" << endl;

    // Loop de entrada dinâmica
```

```

while(true) {
    cin >> entrada;

    if(entrada == -1) {
        break; // Sai do loop se o usuário digitar -1
    }

    leituras_bateria.push_back(entrada); // Adiciona ao final da lista
}

cout << "\nTotal de registros capturados: " << leituras_bateria.size() << endl;

// Verificação de segurança antes de acessar índices específicos
if (leituras_bateria.size() >= 3) {
    cout << "A terceira leitura registrada foi: " << leituras_bateria.at(2) << endl;
}

return 0;
}

```

Overwriting cpp_files/once_exemplo.cpp

Iterando em Vectors

Existem duas formas principais de percorrer um vector. A escolha depende de você precisar ou não do índice (a posição) do elemento.

1. Usando .size() e o loop for tradicional

É a forma clássica, útil quando precisamos saber em qual posição o robô está (ex: "estou no waypoint 3").

2. Usando o Range-based for (For-each)

É a forma mais moderna e limpa. Ela percorre cada elemento automaticamente, sem que você precise gerenciar índices.

```

In [ ]: %%writefile cpp_files/segundo_exemplo.cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<float> velocidades = {0.5, 1.2, 0.8, 1.5};

    // Forma 1: Usando .size()
    cout << "Iterando com .size():" << endl;
    for (int i = 0; i < velocidades.size(); i++) {
        cout << "Velocidade no ponto " << i << ": " << velocidades[i] << endl;
    }
}

```

```

// Forma 2: For-each
cout << "\nIterando com For-each:" << endl;
for (float v : velocidades) {
    cout << "Velocidade: " << v << endl;
}

return 0;
}

```

Segurança e Remoção de Elementos

Em sistemas reais, a robustez do código é prioridade. Antes de realizar qualquer operação de remoção, é fundamental utilizar a função `.empty()`, que retorna true se o vector estiver vazio. Isso impede que o programa tente acessar endereços de memória inválidos, o que causaria o travamento do sistema.

Para a remoção, temos duas ferramentas principais:

- `pop_back()` : Remove o último elemento inserido.
- `.erase()` : Remove um elemento em qualquer posição. Ele utiliza um Iterador (um apontador de posição) que definimos somando um deslocamento ao início do vector (`.begin()`).

```

In [ ]: %%writefile cpp_files/terceiro_exemplo.cpp
#include <iostream>
#include <vector>
#include <string>

using namespace std;

int main() {
    // Nossa lista de afazeres
    vector<string> tarefas = {"Lavar o carro", "Estudar C++", "TAREFA_CANCELADA"};

    cout << "--- Gerenciador de Tarefas ---" << endl;

    // 1. Remocao Especifica: Notei que uma tarefa foi cancelada (indice 2)
    if (!tarefas.empty() && tarefas.size() > 2) {
        cout << "Removendo do sistema: " << tarefas.at(2) << endl;
        tarefas.erase(tarefas.begin() + 2);
    }

    cout << "-----" << endl;

    // 2. Processamento Iterativo: Executa ate a lista ficar vazia
    while (!tarefas.empty()) {
        // .back() acessa o ultimo item para sabermos o que fazer
        cout << "Executando agora: " << tarefas.back() << endl;

        // .pop_back() remove o item que acabamos de concluir
        tarefas.pop_back();
    }
}

```

```

        cout << "Restam " << tarefas.size() << " tarefas na fila." << endl;
    }

    cout << "-----" << endl;
    cout << "Todas as tarefas foram concluidas com segurança!" << endl;

    return 0;
}

```

Matrizes Dinâmicas (vector<vector>)

Uma das maiores vantagens dos vectors é a capacidade de criar matrizes onde o número de linhas e colunas não é fixo. Isso permite que cada "linha" da sua matriz tenha um tamanho diferente, adaptando-se à quantidade de informação disponível.

Imagine que você está organizando a coleta de dados em lotes. Em uma rodada você pode coletar 3 amostras e, na próxima, coletar 10. Com um `vector de vectors`, a estrutura se molda dinamicamente a cada lote, otimizando o uso da memória.

```

In [ ]: %%writefile cpp_files/quarto_exemplo.cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Matriz dinâmica: um vector que armazena outros vectors de inteiros
    vector<vector<int>> lotes_de_dados;

    // Criando o primeiro lote com 3 elementos
    vector<int> lote1 = {10, 20, 30};
    lotes_de_dados.push_back(lote1);

    // Criando o segundo lote com 5 elementos
    vector<int> lote2 = {5, 15, 25, 35, 45};
    lotes_de_dados.push_back(lote2);

    cout << "Total de lotes: " << lotes_de_dados.size() << endl;

    // Acessando dados: lotes_de_dados[indice_do_lote][indice_da_amostra]
    cout << "Amostra [0][1]: " << lotes_de_dados[0][1] << endl;
    cout << "Amostra [1][3]: " << lotes_de_dados[1][3] << endl;

    // Verificando o tamanho dinâmico de cada linha
    for (int i = 0; i < lotes_de_dados.size(); i++) {
        cout << "0 Lote " << i + 1 << " tem " << lotes_de_dados[i].size() <<
    }

    return 0;
}

```

Pairs (std::pair)

O Pair é a estrutura mais simples do C++ para agrupar dois dados que devem caminhar juntos. Em vez de criar duas variáveis separadas, você cria um "par". Esses dados podem ser do mesmo tipo ou de tipos diferentes.

Para acessar os valores, usamos dois comandos simples:

- .first: Acessa o primeiro elemento.
- .second: Acessa o segundo elemento.

```
In [ ]: %%writefile cpp_files/quinto_exemplo.cpp
#include <iostream>
#include <utility> // Biblioteca para o pair
#include <vector>
#include <string>

using namespace std;

int main() {
    // Criando uma lista onde cada item é um PAR (Tarefa, Minutos)
    vector<pair<string, int>> cronograma = {
        {"Revisao doCodigo", 30},
        {"Montagem do Circuito", 60},
        {"Calibracao de Sensores", 15},
        {"Limpeza da Bancada", 10}
    };

    int tempo_total = 0;

    cout << "--- Cronograma de Trabalho ---" << endl;

    // Iterando sobre a lista de pares
    for (const auto& tarefa : cronograma) {
        cout << "- " << tarefa.first << ": " << tarefa.second << " min" << endl;

        // Somando o tempo de cada tarefa ao total
        tempo_total += tarefa.second;
    }

    cout << "-----" << endl;
    cout << "Estimativa total de trabalho: " << tempo_total << " minutos." << endl;

    return 0;
}
```

Maps (std::map)

O Map é uma estrutura de busca baseada em Chave-Valor. Diferente dos vectors, onde você usa um índice numérico, no Map você define uma "etiqueta" (Chave) para encontrar um dado (Valor).

Características Principais:

- Chaves Únicas: Cada etiqueta é exclusiva; se você atribuir um novo valor a uma chave existente, o dado antigo será sobrescrito.
- Busca por Nome: Ideal para organizar informações que não seguem uma ordem numérica lógica.
- Segurança com `.count()` : Antes de acessar uma chave, usamos o `.count(chave)` para verificar se ela existe (retorna 1) ou não (retorna 0). Isso é vital porque tentar acessar uma chave inexistente com `[]` faz o C++ criar um item vazio no mapa por acidente.

Iteração com Structured Bindings

A forma mais moderna de percorrer um mapa é o Structured Bindings (C++17). Em vez de lidarmos com ponteiros complexos, "desempacotamos" o par chave-valor diretamente no loop.

```
In [ ]: %%writefile cpp_files/sexta_exemplo.cpp
#include <iostream>
#include <map>
#include <string>

using namespace std;

int main() {
    // Chave: Nome do Produto | Valor: Quantidade em estoque
    map<string, int> estoque;

    // Inserção de dados
    estoque["Caderno"] = 15;
    estoque["Caneta"] = 25;
    estoque["Borracha"] = 10;

    // Verificação de Segurança
    string busca = "Regua";
    if (estoque.count(busca)) {
        cout << busca << " em estoque: " << estoque[busca] << endl;
    } else {
        cout << "Aviso: " << busca << " nao cadastrada." << endl;
    }

    cout << "\n--- Inventario Completo ---" << endl;
```

```
// Iteração moderna: [chave, valor]
// auto const& garante performance (referência) e segurança (const)
for (auto const& [item, qtd] : estoque) {
    cout << "Item: " << item << " | Quantidade: " << qtd << endl;
}

return 0;
}
```

Repare no uso de auto const& no loop. Isso evita que o C++ faça uma cópia de cada item do mapa a cada volta do loop, economizando memória e tornando o processamento muito mais rápido.

Faça a lista de exercícios desta aula e não hesite em tirar dúvidas!